PROJECT REPORT

DEVELOPMENT OF A CONSOLE-BASED NUMBER GUESSING GAME USING C PROGRAMMING

A Beginner-Level Programming Project

Submitted By:	_____
UID / Roll Number:	_____
Course:	BCA Data Science
University:	Chandigarh University
Subject:	C Programming
Submitted To:	_____
Academic Session:	_____

Department of _____

TABLE OF CONTENTS

1	Introduction
2	Objectives of the Project
3	Project Description
4	Hardware and Software Requirements
5	Programming Concepts Used
6	Algorithm
7	Flowchart
8	Source Code
9	Explanation of the Program
10	Sample Output
11	Testing and Results
12	Advantages
13	Limitations
14	Future Scope
15	Conclusion
16	References

1. INTRODUCTION

The Number Guessing Game is a simple, interactive console-based application developed using the C programming language. In this game, the computer selects a random number within a fixed range, and the user tries to identify that number by entering guesses.

After each guess, the program gives a hint. If the entered number is smaller than the hidden number, the program displays “Too low”. If it is greater, the program displays “Too high”. The game continues until the user enters the correct number.

This project demonstrates how basic programming concepts can be combined to create a working application. It is suitable for beginners because it uses variables, input/output, conditional statements, loops and library functions.

2. OBJECTIVES OF THE PROJECT

• To develop an interactive game using the C programming language.

• To generate a random number within a specified range.

• To accept and process input entered by the user.

• To apply conditional statements to compare two numbers.

• To use a loop so the game continues until the correct guess is entered.

• To count and display the number of attempts.

• To improve logical thinking, debugging and problem-solving skills.

3. PROJECT DESCRIPTION

The program generates a secret integer from 1 to 100. The user is asked to enter a guess. The program compares the guess with the secret number and displays an appropriate hint. The number of attempts is increased after every guess.

When the user guesses correctly, the program displays a success message along with the total number of attempts. The program then ends. Each new execution generates a new pseudo-random sequence, so the answer will normally change every time the game is played.

4. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

• Desktop computer or laptop.

• Keyboard for entering guesses.

• Monitor or display screen.

• At least 2 GB RAM; higher specifications may be used.

Software Requirements

• Operating system: Windows, Linux or macOS.

• Programming language: C.

• Compiler: GCC or another standard C compiler.

• IDE or editor: Visual Studio Code, Code::Blocks, Dev-C++ or another compatible editor.

5. PROGRAMMING CONCEPTS USED

5.1 Header Files

The program includes stdio.h for input and output functions, stdlib.h for rand() and srand(), and time.h for obtaining the current time.

5.2 Variables and Data Types

Integer variables store the generated number, the user's guess and the attempt count. The int data type is appropriate because the game uses whole numbers.

5.3 Random Number Generation

The rand() function generates a pseudo-random integer. The expression rand() % 100 + 1 converts the result into a number from 1 to 100. The srand(time(0)) statement seeds the generator using the current time so that different program runs normally produce different sequences.

5.4 Conditional Statements

An if-else if-else structure checks whether the guess is lower than, higher than or equal to the secret number.

5.5 Loops

A do-while loop ensures that the user is asked to guess at least once. It repeats while the guess is not equal to the secret number.

5.6 Input and Output

The printf() function displays instructions and hints. The scanf() function reads the integer entered by the user.

6. ALGORITHM

Step 1: Start the program.

Step 2: Include the required header files.

Step 3: Initialize the random number generator using srand(time(0)).

Step 4: Generate a random number between 1 and 100.

Step 5: Set the attempt counter to zero.

Step 6: Ask the user to enter a guess.

Step 7: Read the guess and increase the attempt counter.

Step 8: Compare the guess with the secret number.

Step 9: If the guess is smaller, display “Too low”.

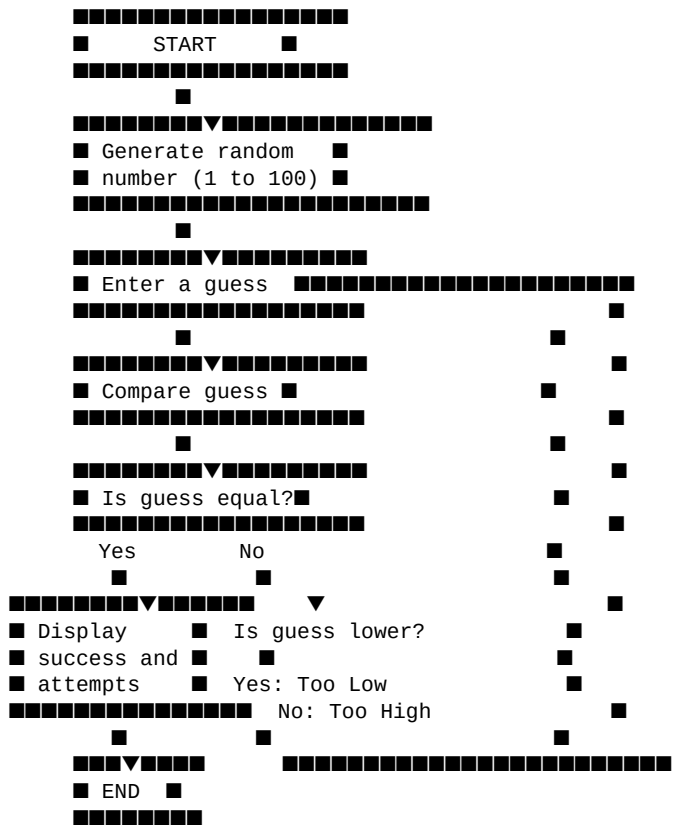
Step 10: If the guess is greater, display “Too high”.

Step 11: If the guess is correct, display a congratulatory message and the number of attempts.

Step 12: Stop the program.

7. FLOWCHART

The following flowchart represents the main logic of the game:



8. SOURCE CODE

The following C program implements the Number Guessing Game. It uses a range of 1 to 100 and counts the guesses made by the user.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    int number, guess, attempts = 0;

    /* Initialize random number generator */
    srand(time(0));

    /* Generate a number between 1 and 100 */
    number = rand() % 100 + 1;

    printf("================================\n");
    printf("       NUMBER GUESSING GAME      \n");
    printf("================================\n");
    printf("I have selected a number from 1 to 100.\n");
    printf("Try to guess the number!\n\n");
```

```
    do
    {
        printf("Enter your guess: ");

        if (scanf("%d", &guess) != 1)
        {
            printf("Invalid input. Please enter a whole number.\n");
            return 1;
        }

        attempts++;

        if (guess < number)
            printf("Too Low! Try Again.\n\n");
        else if (guess > number)
            printf("Too High! Try Again.\n\n");
        else
        {
            printf("\nCongratulations! You guessed correctly.\n");
            printf("The correct number was %d.\n", number);
            printf("Total attempts: %d\n", attempts);
        }

    } while (guess != number);

    return 0;
}
```

Note: The input check prevents the program from continuing when the user enters non-numeric input. A further improvement could reject numbers outside the range 1 to 100.

9. EXPLANATION OF THE PROGRAM

Step 1: Header Files

The program includes stdio.h, stdlib.h and time.h. These provide the functions required for displaying messages, generating random numbers and using the current time.

Step 2: Variable Declaration

The variables number and guess store the secret number and the user's input. The attempts variable counts how many guesses have been entered and starts at zero.

Step 3: Random Number Generation

srand(time(0)) initializes the pseudo-random number generator. The expression rand() % 100 + 1 produces a number in the range 1 to 100.

Step 4: Taking User Input

The program displays instructions and asks the user to enter a guess. scanf() reads the entered integer.

Step 5: Comparing the Guess

The if-else if-else statements compare the user's guess with the generated number. A hint is displayed depending on whether the guess is too low, too high or correct.

Step 6: Counting Attempts

The attempts variable is increased by one each time the user enters a valid integer.

Step 7: Ending the Game

The do-while loop repeats until the guess matches the secret number. After a correct guess, the program displays the result and returns zero to indicate successful completion.

10. SAMPLE OUTPUT

The output below is an example. The secret number is generated randomly, so the actual number and guesses may differ each time the program runs.

```
================================
       NUMBER GUESSING GAME
================================
I have selected a number from 1 to 100.
Try to guess the number!

Enter your guess: 40
Too Low! Try Again.

Enter your guess: 75
Too High! Try Again.

Enter your guess: 60
Too Low! Try Again.

Enter your guess: 68

Congratulations! You guessed correctly.
The correct number was 68.
Total attempts: 4
```

11. TESTING AND RESULTS

The program was tested with different guesses to check whether the comparison logic and attempt counter work correctly.

Test case	Input / situation	Expected result
1	Guess is lower than secret number	Display “Too Low! Try Again.”
2	Guess is higher than secret number	Display “Too High! Try Again.”
3	Guess equals secret number	Display success message and total attempts
4	User enters letters	Display invalid-input message and stop
5	Run the program again	Generate a new pseudo-random sequence

Result: The program successfully generates a number, accepts guesses, provides useful hints and terminates when the correct number is entered. The attempt counter displays the number of guesses made.

12. ADVANTAGES

• Simple and easy to understand for beginners.

• Provides an interactive experience through the console.

• Demonstrates important C programming concepts.

• Uses very little memory and processing power.

• Can be played again by running the program.

• Helps improve logical thinking and problem-solving skills.

13. LIMITATIONS

• The game has a fixed number range of 1 to 100.

• There is no graphical user interface; interaction takes place in the console.

• The game does not save previous scores or player records.

• There are no difficulty levels or time limits in the basic version.

• The random number generator produces pseudo-random numbers, not truly unpredictable random values.

14. FUTURE SCOPE

The project can be improved by adding more features in future versions:

• Difficulty levels: Easy, Medium and Hard with different number ranges.

• A limited number of attempts to make the game more challenging.

• A replay option to start a new round without restarting the program.

• A scoring system that awards more points for fewer guesses.

• A high-score system that stores the best result in a file.

• Input validation to reject numbers outside the selected range.

• A graphical user interface for a more attractive gaming experience.

15. CONCLUSION

The Number Guessing Game is a basic but useful project developed using the C programming language. It demonstrates how random number generation, variables, loops, conditional statements and input/output functions work together in a practical application.

Through this project, the concepts of C programming become easier to understand because they are applied to a simple interactive game. The program successfully allows the computer to generate a random number and enables the user to guess it with the help of hints.

Overall, this project improves programming knowledge, logical thinking and problem-solving ability. It can also be expanded with additional features such as difficulty levels, scoring and file handling.

16. REFERENCES

• The C Programming Language, Brian W. Kernighan and Dennis M. Ritchie.

• C standard library documentation: stdio.h, stdlib.h and time.h.

• Programiz – C Programming tutorials: https://www.programiz.com/c-programming

• GeeksforGeeks – C Programming resources: https://www.geeksforgeeks.org/c/
• Class notes and learning materials provided during the C Programming course.